

De los productos con interfaz a las APIs de Claude

Claude Certified Developer, Foundation (CCDV-F) · Masterclass UST

ES

Sesión 4 de 12 · Notas en español (es-ES)

Tras el repaso de productos, la sesión 4 trató las APIs en dos capas que en Foundation se mezclan: (1) el **contrato OpenAPI** de la app que generáis (REST, Swagger UI, design-first), y (2) el hecho de que Claude Code es un cliente que más adelante se reconstruye con las **APIs de Claude / Agent SDK**. Este capítulo es la primera capa, y cómo los clientes con GUI (Code, Cowork Design) producen ese contrato y un backend ejecutable sin que tecleéis los handlers.

LOS DOCS SE MUEVEN; LOS CONTRATOS PERMANECEN

OpenAPI 3.1, las rutas de Swagger UI y los nombres de carpetas de skills del demo pueden haber cambiado. El examen pide que expliquéis APIs design-first, cómo una skill inyecta el rol de arquitecto de API, y que adjuntar un YAML es más barato que adjuntar todo el backend. Confirmad la **visión general de la API** cuando lleguéis a los módulos 07+.

OpenAPI design-first, no «código y luego documentar»

El formador situó como primer artefacto que un equipo full-stack debe congelar una **especificación OpenAPI** (mencionó W3C/CNCF de pasada; en la práctica el demo usa OpenAPI, p. ej. 3.1, con documentación estilo Swagger). El método es **design-first**: escribir el contrato, circularlo a backend y frontend, e implementar. Es lo contrario de generar código y esperar que la documentación alcance.

- Rol: arquitecto / diseñador de API — no es la misma skill que el «principal solution architect», aunque ambos consumen el SRS.
- Contenido: rutas, modelos de petición/respuesta, autenticación (luego el demo necesitó un secreto JWT en `.env`), GET frente a POST frente a PUT según el arquitecto.
- Salida en el laboratorio: `openapi.yaml` en el repo, y backends generados *a partir de* ese fichero, no de un prompt vago de «monta una API».
- El frontend usa el mismo YAML (o tipos generados) para que las pantallas no inventen campos que la API no tiene.

Una skill OpenAPI es un rol invocado con slash

Como las skills de analista y arquitecto, la de OpenAPI vive como `SKILL.md` en una carpeta con nombre (el formador creó un slash de «open API»). El cuerpo dice: actúa como arquitecto de API, sigue OpenAPI 3.1, produce documentación Swagger, modelos de request/response, recursos con forma de microservicio si esa es la arquitectura. Los verbos HTTP no son trivia: si el arquitecto quiere PATCH en vez de PUT, eso va en el spec y en la skill, no como sorpresa en el Express/FastAPI generado.

Adjuntáis el BRD/SRS (o HLD) al prompt, invocáis el slash y pedís YAML en una carpeta conocida (p. ej. docs/). Refrescad la lista de skills si no aparece el slash nuevo. Ese YAML pasa a ser el adjunto preferido de las skills siguientes — más barato que volver a adjuntar todo backend/.

Backend desde el contrato: FastAPI, Node o lo que diga la memoria

Una **skill de backend** más un CLAUDE .md acotado al backend dicen a Claude Code qué runtime usar (el demo usó estilo Node: package.json, npm run build / npm start, puerto 3000). La instrucción «implementa solo las APIs, desde este OpenAPI» evita un esquema paralelo inventado. La memoria anidada importa: el orquestador puede cargar el CLAUDE.md global más backend/CLAUDE.md para esa subtask (el módulo 05 profundiza en el reparto de agentes).

1. Sesión nueva para una actividad nueva, para que el contexto no mezcle documentos de arquitectura con ruido de implementación.
2. Slash de backend; adjuntar openapi.yaml; nombrar la carpeta de salida.
3. Dejar correr el bucle de cinco fases: aclarar, planificar (carpetas, controladores, servicios), ejecutar, verificar (tests — 43 en verde en el demo), entregar un resumen.
4. Ejecutadlo vosotros o pedídselo a Claude. Faltar .env (secreto JWT) es un primer fallo típico: es configuración, no un bug del modelo.
5. Si Swagger UI no estaba en la skill, un segundo pase («incluye Swagger UI») funciona pero viene a *doblar* tokens porque el modelo relee el árbol. Metedlo en CLAUDE.md o en la skill a la primera.

IMPUESTO DE VERIFICACIÓN

Claude Code puede intentar demostrar código limpio y relanzar tests en cada fichero. Esa es la fase «verify». Desactivadla o aplazadla en memoria si vais a hacer smoke test en local. El formador aun así usó los tests como listón de calidad al añadir Swagger — 43 casos — así que «saltar verify» es control de coste, no una afirmación de que los tests no importan.

Qué significa «APIs de Claude» en Foundation a partir de aquí

Dos frases de la sesión que no debéis fusionar: *implementaremos herramientas MCP propias cuando implementemos APIs*, y *Claude Code ya es un cliente agentic sobre las APIs de Claude*. La sesión 4 no instala el SDK de Python/TS. Muestra que los productos con GUI no son un callejón: ya llaman a modelos con tools, guardan resultados en un sandbox y pueden generar servicios REST que luego abríis en el navegador. Los módulos 07+ hacen explícita la Messages API. Aquí, el movimiento de examen es: cliente GUI → artefactos (OpenAPI, código) → una API HTTP real que podéis consultar con curl o Swagger.

Prueba de aula de que el backend es real

Tras `npm run build` y `npm start`, Swagger UI listó `register / login / logout / refresh` y rutas con candado. Try-it-out creó un usuario, el login devolvió un bearer, `/me` (o equivalente) autorizado devolvió el usuario. No se tecleó código de aplicación. Los fallos (host incorrecto, secreto ausente, Swagger no generado a la primera) son ingeniería ordinaria, no «el LLM se negó».

Cowork Design como otro camino GUI hacia las mismas APIs

Para sembrar el frontend, el formador pasó a **Cowork** → Design (abajo a la izquierda en la UI de entonces) y pidió wireframes de la app de tareas. Adjuntó el YAML OpenAPI en vez de toda la carpeta backend para ahorrar tokens. Un documento de marca/logo habría sido el otro adjunto de alto valor. Aparecieron preguntas de la fase aclarar (escritorio frente a responsive, personal frente a empresa, Kanban frente a lista): responded de forma estrecha si estáis racionando tokens (escritorio + personal en el demo).

Un alumno señaló que en la industria a menudo se diseña la UI primero y luego el esquema. El orden del laboratorio es el inverso a propósito: el YAML ya lista los campos; Design lee esquemas y propone pantallas (login, registro, dashboard, CRUD). Aun así podéis importar HTML a Figma (plugins HTML-to-design) si UX va a pulir ahí. Editar en Cowork Design es «un poco caro»; Figma es para personas que iteran en visual. Ese fichero de Figma será la entrada MCP del capítulo siguiente.

Postura de examen

Foundation no es «escribir FastAPI a mano». Es: saber que OpenAPI design-first es el lenguaje compartido entre agentes, que skills y CLAUDE.md dirigen la generación, que un segundo pase de generación es una penalización de tokens, y que Chat/Cowork/Code son cómo llegáis a los modelos hasta que llaméis vosotros a la Messages API. Dejad los nombres de producto en inglés en las respuestas escritas.

Documentación oficial

- **Visión general de la API**

<https://docs.claude.com/en/api/overview>

Hacia dónde van los clientes GUI: Claude programático, no solo Chat.

- **Messages API**

<https://docs.claude.com/en/api/messages>

La forma de petición que implementan módulos posteriores; Code ya la envuelve.

- **Visión general de Claude Code**

<https://code.claude.com/docs/en/overview>

Skills, memoria y ejecución de servicios generados desde el agente.

- **Workspaces**

<https://docs.claude.com/en/docs/build-with-claude/workspaces>

Acceso a modelos a nivel de organización, que limita qué APIs y modelos podéis llamar.

Study notes distilled from the UST CCD Foundation masterclass. Prefer Anthropic's current docs over session anecdotes. Not official exam material.